

Mảng Một Chiều & Xử Lý Dãy Số (1D Arrays)

Chương 1 • Bài 4 • Khóa Học Lập Trình Thi Đấu Cơ Bản THPT

NỘI DUNG BÀI HỌC

- Phần 1: Khái niệm và tại sao cần sử dụng mảng một chiều?
- Phần 2: Khai báo mảng tĩnh trong C++ và danh sách (list) trong Python.
- Phần 3: Nhập xuất và kỹ thuật duyệt mảng hiệu quả.
- Phần 4: Các bài toán mẫu kinh điển: Tìm Max/Min, tính tổng, đếm phần tử.
- Phần 5: Kỹ thuật Mảng đánh dấu (Frequency Array / Counting).
- Phần 6: Các lỗi sai sót thường gặp và giới hạn kích thước mảng.

1. Tại sao cần mảng một chiều?

Khi cần quản lý nhiều giá trị có cùng kiểu dữ liệu (ví dụ: điểm của 10,000 học sinh, tọa độ của các đỉnh, giá trị dãy số), ta không thể khai báo từng biến riêng lẻ như $a_1, a_2, \dots, a_n$. Mảng một chiều là tập hợp liên tiếp các ô nhớ lưu trữ các phần tử cùng kiểu, được truy xuất thông qua chỉ số (index) với thời gian hằng số $O(1)$.

2. Khai báo và khởi tạo mảng

Trong lập trình thi đấu (Competitive Programming), các bài toán thường yêu cầu mảng có kích thước N lên tới 10^5 hoặc 10^6 . Việc khai báo đúng vị trí trong bộ nhớ là vô cùng quan trọng.

Ngôn ngữ	Cách khai báo	Vị trí tối ưu	Lưu ý
C++ (Mảng tĩnh)	<code>int a[100005];</code>	Biến toàn cục (Global)	Tự động gán 0, tránh tràn ngăn xếp (Stack)
C++ (Vector)	<code>vector a(n);</code>	Cục bộ trong hàm main	Tự co giãn, an toàn, hỗ trợ nhiều hàm STL
Python (List)	<code>a = [0] * n</code>	Trong script	Khởi tạo nhanh danh sách n phần tử 0
Python (Đọc nhanh)	<code>a = list(map(int, input().split()))</code>	Khi nhập mảng	Đọc toàn bộ dãy số trên 1 dòng

[C++]

```
// C++: Khai báo mảng toàn cục tránh tràn Stack
#include <iostream>
#include <vector>
using namespace std;

const int MAXN = 100005;
int a[MAXN]; // Mảng toàn cục lưu trữ tối đa 100,000 phần tử

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    if (!(cin >> n)) return 0;
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }
    return 0;
}
```

3. Các bài toán kinh điển trên mảng một chiều

3.1. Tìm phần tử lớn nhất (Max) và nhỏ nhất (Min)

Để tìm Max/Min, khởi tạo biến kết quả bằng giá trị của phần tử đầu tiên $a[0]$, sau đó duyệt qua các phần tử còn lại từ 1 đến $n-1$ và cập nhật nếu tìm thấy giá trị lớn hơn hoặc nhỏ hơn.

[C++]

```
// Tìm phần tử lớn nhất và nhỏ nhất
int max_val = a[0];
int min_val = a[0];
for (int i = 1; i < n; i++) {
    if (a[i] > max_val) max_val = a[i];
    if (a[i] < min_val) min_val = a[i];
}
cout << "Max: " << max_val << ", Min: " << min_val << "\n";
```

3.2. Tính tổng các phần tử và tránh tràn số

Nếu mảng có $N = 10^5$ phần tử và mỗi phần tử $a[i]$ có thể đạt 10^9 , tổng lớn nhất có thể lên tới 10^{14} . Kiểu số nguyên thông thường int trong C++ chỉ biểu diễn tối đa $\sim 2 \times 10^9$, do đó sẽ xảy ra lỗi tràn số nghiêm trọng. Bắt buộc phải sử dụng biến tổng kiểu long long.

[C++]

```
// Tính tổng mảng an toàn không tràn số
long long sum = 0;
for (int i = 0; i < n; i++) {
    sum += a[i];
}
cout << "Tong: " << sum << "\n";
```

4. Kỹ thuật Mảng đánh dấu (Frequency Array)

Khi cần đếm số lần xuất hiện của từng số trong dãy số hoặc kiểm tra xem một số x có tồn tại trong dãy hay không, kỹ thuật mảng đánh dấu là lựa chọn số 1 với thời gian tìm kiếm $O(1)$. Gọi $\text{cnt}[v]$ là số lần xuất hiện của giá trị v .

[C++]

```
// Đếm tần suất xuất hiện với mảng cnt
const int MAX_VAL = 1000005;
int cnt[MAX_VAL] = {0}; // Mọi phần tử ban đầu có tần suất bằng 0

for (int i = 0; i < n; i++) {
    cnt[a[i]]++; // Ghi nhận giá trị a[i] xuất hiện thêm 1 lần
}

// Truy vấn tần suất của số x trong O(1)
int x;
cin >> x;
cout << "Số " << x << " xuất hiện: " << cnt[x] << " lần\n";
```

CÁC BẦY KINH ĐIỂN CẦN TRÁNH:

1. Khai báo mảng cục bộ quá lớn: Khai báo `int a[1000000]` bên trong hàm `main()` sẽ gây lỗi tràn bộ nhớ ngăn xếp (Stack Overflow) làm chương trình thoát đột ngột (Runtime Error / Crash). Luôn khai báo mảng lớn ngoài hàm `main` (biến toàn cục) hoặc dùng `vector`.
2. Truy cập ngoài biên (Out of Bounds): Mảng có N phần tử có chỉ số từ 0 đến $N-1$ (hoặc 1 đến N). Truy cập `a[N]` khi mảng chỉ số 0.. $N-1$ dẫn tới giá trị rác hoặc lỗi bộ nhớ.
3. Tràn kiểu khi tính tổng: Luôn dùng `long long` cho biến cộng dồn tổng các phần tử.